

Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 6

Lab Activity: Threads: Introductory: Lost-Update 1

Understanding the 'lost-update' problem

1. Inspect the initial value of the data field

Q1. If one thousand increment updates are performed, what value should it end up at? 2000

2. Click the 'Start ADD Thread' button to run the ADD thread in isolation.

Q2. Was the result as expected? (if not, check your maths!).

Threads and Locking

The 'ADD' Thread

The ADD thread invokes one thousand update transactions on the data field.
Each transaction comprises the sequential actions: READ value; UPDATE value; WRITE value.
The UPDATE action is a simple

Transaction

READ data field into local variable (assume initial value is
UPDATE local variable (100 +1 = 101),
WRITE value from local variable to data

Start ADD Thread

Transaction count

1000

The 'SUBTRACT' Thread

The SUBTRACT thread invokes one thousand update transactions on the data field.
Each transaction comprises the sequential actions: READ value; UPDATE value; WRITE value.
The UPDATE action is a simple

Transaction

READ data field into local variable (assume initial value is
UPDATE local variable (100 -1 = 99),
WRITE value from local variable to data

Start SUBTRACT Thread

Transaction count

0

Start both threads
(threads will run concurrently)

Done

Locking configuration (both threads)

☒ No Locking
☐ Use Locking

APPLY LOCK

☐ Before READ
☒ After READ and Before UPDATE
☐ After UPDATE and Before WRITE

RELEASE LOCK

☐ After READ and Before UPDATE
☒ After UPDATE and Before WRITE
☐ After WRITE

Data field (a shared resource)

2000

Reset data field
(to 1,000)

3. Note the value of the data field now.

Q3. If one thousand decrement updates are performed, what value should it end up at? 1000

- Click the 'Start SUBTRACT Thread' button to run the SUBTRACT thread in isolation.

Q4. Was the result as expected? (if not, check your maths!).

Threads and Locking

The 'ADD' Thread

The ADD thread invokes one thousand update transactions on the data field.
Each transaction comprises the sequential actions: READ value; UPDATE value; WRITE value.
The UPDATE action is a simple

Transaction

READ data field into local variable (assume initial value is
UPDATE local variable (100 +1 = 101),
WRITE value from local variable to data

Start ADD Thread

Transaction count

The 'SUBTRACT' Thread

The SUBTRACT thread invokes one thousand update transactions on the data field.
Each transaction comprises the sequential actions: READ value; UPDATE value; WRITE value.
The UPDATE action is a simple

Transaction

READ data field into local variable (assume initial value is
UPDATE local variable (100 -1 = 99),
WRITE value from local variable to data

Start SUBTRACT Thread

Transaction count

1000

Start both threads
(threads will run concurrently)

Done

Locking configuration (both threads)

☒ No Locking
☐ Use Locking

APPLY LOCK

☐ Before READ
☒ After READ and Before UPDATE
☐ After UPDATE and Before WRITE

RELEASE LOCK

☐ After READ and Before UPDATE
☒ After UPDATE and Before WRITE
☐ After WRITE

Data field (a shared resource)

1000

Reset data field
(to 1,000)

Q5. If the threads are run sequentially (as above), are there any issues concerning access to the shared variable that can lead to its corruption? No. Because there is only one thread access the variable now.

- Click the 'Reset data field' button to reset the value of the data variable to 1000.

Q6. Given a starting value of 1000, if one thousand increment updates are performed, and one thousand decrements are performed, what should the final value be? $1000 + 1000 - 1000 = 1000$

Threads and Locking

The 'ADD' Thread

The ADD thread invokes one thousand update transactions on the data field.
Each transaction comprises the sequential actions: READ value; UPDATE value; WRITE value.
The UPDATE action is a simple

Transaction

READ data field into local variable (assume initial value is
UPDATE local variable (100 +1 = 101),
WRITE value from local variable to data

Start ADD Thread

Transaction count

1000

The 'SUBTRACT' Thread

The SUBTRACT thread invokes one thousand update transactions on the data field.
Each transaction comprises the sequential actions: READ value; UPDATE value; WRITE value.
The UPDATE action is a simple

Transaction

READ data field into local variable (assume initial value is
UPDATE local variable (100 -1 = 99),
WRITE value from local variable to data

Start SUBTRACT Thread

Transaction count

1000

Locking configuration (both threads)

☒ No Locking
☐ Use Locking

APPLY LOCK

☐ Before READ
☒ After READ and Before UPDATE
☐ After UPDATE and Before WRITE

RELEASE LOCK

☐ After READ and Before UPDATE
☒ After UPDATE and Before WRITE
☐ After WRITE

Data field [a shared resource]

1066

Reset data field (to 1,000)

Start both threads (threads will run concurrently)

Done

6. Click the 'Start both threads' button to run the two threads concurrently.
7. When both threads have finished check their transaction counts to ensure both executed 1000 transactions, hence determine the correct value of the data variable.

Q8. Is the actual data variable value correct? If not what could have caused the discrepancy?

Error due to lost-update because:

- Thread 1 reads the value at time A
- Thread 2 also reads the value at time A
- Increment/decrement and overwrite each other -> Lost the update.

Lab Activity: Threads: Introductory: Lost-Update 1

Understanding the 'lost-update' problem

The Lost-Update 1 activity exposed the lost-update problem.

We saw that a discrepancy occurred between the expected value and the actual final value of a shared variable.

Some of the updates were lost. This is because one thread was allowed to access the variable whilst the other had already started a transaction based on the value of the variable.

Q1. How does this problem relate to transactions in database applications?

It is identical to the lost-update anomaly in databases where multiple transactions update the same data without proper locking.

Q2. How many of the ACID properties of transactions have been violated?

Property	Violated?	Reason
Atomicity	Yes	Update is not guaranteed as a whole
Isolation	Yes	Transactions interfere with each other
Consistency	Potentially	Final data becomes incorrect
Durability	No	Not relevant here

→ Violates: A and I (and can affect C indirectly)

Q3. How could this problem affect applications such as: An on-line flight booking system? A warehouse stock-management system?

System	Impact
Online flight booking	Two users can book the same seat
Warehouse stock system	Wrong stock levels → overselling or negative inventory

Q4. Think of at least one other real-world application that would be affected by the lost-update problem?

- Banking: Two people withdraw money at the same time → incorrect balance.
- Online payments, e-wallets, ticketing, inventory, gaming resources, etc.

Q5. Compare the three discrepancies. Is the actual discrepancy predictable, or does it seem to have a random element? Random

Q6. If there is a random element, where does it come from (think carefully about the mechanism at play here – two threads are executing concurrently but without synchronization – what could go wrong)?

- CPU scheduling is unpredictable.
- The order of thread execution changes each run.

Q7. Is the problem more serious if the discrepancy is predictable? Or if the discrepancy is not predictable?

- Unpredictable discrepancies are more dangerous because:
- They are hard to test, detect, and debug.
 - You cannot correct them with simple logic.

Lab Activity: Threads: Introductory: Locks 1 The need for locks

The Lost-Update 1 and Lost-Update 2 activities exposed the lost-update problem, and showed us that the extent of the problem is unpredictable. Therefore, we must find a mechanism to prevent the problem from occurring.

Q1. Could a locking mechanism be the answer? If so, how would it work?

Yes.

A locking mechanism can prevent the lost-update problem by ensuring that only one thread at a time is allowed to read and write to the shared variable.

When a thread needs to perform a transaction:

1. It acquires a lock before accessing the shared variable.
2. It completes its read–modify–write operation.
3. It releases the lock, allowing the next thread to proceed.

This guarantees that no other thread can interfere while one update is in progress.

Q2. Would it be adequate to apply the lock to only one of the threads, or does it have to apply to both?

It must apply to both threads.

If only one thread uses a lock, the other thread can still access the variable at the same time, causing lost updates. All threads accessing the shared resource must respect the same locking rule.

Q3. Is it necessary to use a read-lock, a write-lock, or is it important to prevent both reading and writing while the lock is applied, to ensure that no lost-updates occur (think about the properties of transactions)?

We must **prevent both reading and writing while the lock is applied**.

Because:

If a thread is allowed to read while another is writing, it may read an outdated or partial value, leading to incorrect updates. A proper transaction requires:

- The read,
- The modification,
- The write,

to occur as **one atomic block** under the lock.

Therefore, **a mutual exclusion (mutex) lock is needed**, not a separate read-only lock.

Q4. At what point in this sequence should the lock be applied to the transaction? The lock must be applied before the read step. Otherwise, a thread could read a value that is already being modified by another thread, causing lost updates later.

Q5. At what point should the lock be released? Investigate the various combinations of locking strategies:

1. Click the 'Use Locking' button.
2. Repeat the following steps until all combinations of lock apply and release have been tried:
 - A. Ensure the value of the data variable is 1000 (Click the 'Reset data field' button if necessary).
 - B. Select an option from the APPLY LOCK choices.
 - C. Select an option from the RELEASE LOCK choices.
 - D. Click the 'Start both threads' button to run the two threads concurrently.
 - E. When both threads have finished check their transaction counts to ensure both executed 1000 transactions, hence determine the correct value of the data variable.
 - F. Make a note of the extent of the discrepancy.

The lock should be released after the write step. This ensures that the full transaction (read + modify + write) is protected and no other thread interferes.

Q6. Have you found a combination of applying and releasing locks that always ensures that there is NO discrepancy? Are you sure – repeat the emulation with these settings a few times – does it ALWAYS work?

Yes.

The correct strategy is to:

- **Apply the lock before reading,** and
- **Release it after writing.**

If you repeat the experiment multiple times with this combination, the final value will always be correct (1000), proving that the lost-update problem has been eliminated.

The use of the lock forces mutually exclusive access to the variable.

Q7. Could you explain clearly to a friend what the above statement means – try it?

Imagine a single bathroom with a locked door.

Only one person can enter at a time. If the door is locked, others must wait outside until it is unlocked.

Similarly, when a thread locks the shared variable, all other threads are forced to wait. They cannot read or write until the first thread finishes and releases the lock. This is mutual exclusion: only one thread can “use” the shared resource at any moment.

Mutual exclusion prevents the lost-update problem.

Q7. Could you explain clearly to a friend what the above statement means – try it?

Lost updates happen when two threads read and modify a value at the same time, and their results overwrite each other.

Mutual exclusion ensures that only one thread can read/modify/write the value at a time, so:

- No two threads can work on the same value simultaneously.
- No update is overwritten or lost.

Therefore, mutual exclusion = no lost updates.